



1. Problem Title & Link

Title: LeetCode 424 : Longest Repeating Character Replacement

Link: [LeetCode 424 - Longest Repeating Character Replacement](#)

2. Problem Statement (Short Summary)

You are given:

s

k

You can replace at most k characters in the string.

Find the length of the longest substring that can become a string containing only one repeating character after replacements.

In simple terms:

- Change at most k characters
- Make all characters inside a window identical
- Return the maximum possible window length

3. Examples (Input → Output)

Example 1

Input:

s="ABAB"

k=2

Output:

4

Explanation:

Replace:

A → B

B → A

Entire string becomes same.

Example 2

Input:

s="AABABBA"

k=1

Output:

4



Explanation:

Longest valid substring:

"AABA"

Replace:

B → A

Length:

4

Example 3

Input:

s="AAAA"

k=2

Output:

4

4. Constraints

$1 \leq s.length \leq 10^5$

$0 \leq k \leq s.length$

s contains uppercase English letters

Special restrictions:

- Need optimal solution
- Avoid checking all substrings

5. Core Concept (Pattern / Topic)

Sliding Window + Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate every possible substring:

1. Calculate frequencies
2. Determine replacements needed
3. Track maximum length

Time:

$O(n^2)$

Too slow.



Optimized Thinking

Observation:

For a window:

$\text{windowLength} - \text{maxFrequency}$

gives:

number of replacements needed

If:

$\text{windowLength} - \text{maxFrequency}$

$\leq k$

Window is valid.

Otherwise:

Shrink from left.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

AABABBA

$k=1$

Window growth:

A

AA

AAB

AABA

AABAB

Check:

Length=5

Max frequency=3

Needed replacements:

$5-3=2$

Since:

$2 > 1$

Shrink window.

Continue.

Maximum:



4

8. Pseudocode (Language Independent)

left=0

maxFreq=0

answer=0

frequency[26]

for right in range(n):

 add current character

 maxFreq=
 max(maxFreq,
 currentFrequency)

 while windowLength
 -maxFreq
 >k:

 remove left character

 left++

 answer=
 max(answer,
 windowLength)

return answer

9. Code Implementation

Python

class Solution:



```
def characterReplacement(  
    self,  
    s,  
    k):  
  
    freq=[0]*26  
  
    left=0  
    maxFreq=0  
    result=0  
  
    for right in range(len(s)):  
  
        index=ord(  
            s[right]  
        )-ord('A')  
  
        freq[index]+=1  
  
        maxFreq=max(  
            maxFreq,  
            freq[index]  
        )  
  
        while (  
            right-left+1  
            -maxFreq  
            >k  
        ):  
  
            freq[  
                ord(  
                    s[left]  
                )-ord('A')  
            ]-=1  
  
            left+=1
```



```
        result=max(  
            result,  
            right-left+1  
        )  
  
    return result
```

Java

```
class Solution {  
  
    public int characterReplacement(  
        String s,  
        int k) {  
  
        int[] freq=  
            new int[26];  
  
        int left=0;  
        int maxFreq=0;  
        int result=0;  
  
        for(int right=0;  
            right<s.length();  
            right++){  
  
            int index=  
                s.charAt(right)  
                -'A';  
  
            freq[index]++;  
  
            maxFreq=  
                Math.max(  
                    maxFreq,  
                    freq[index]  
                );
```



```
while(
    (right-left+1)
    -maxFreq
    >k
){

    freq[
        s.charAt(left)
        -'A'
    ]--;

    left++;
}

result=
    Math.max(
        result,
        right-left+1
    );
}

return result;
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

Every character enters and exits the window at most once.

Space Complexity:

$O(1)$

Reason:

Frequency array size always remains:

26



11. Common Mistakes / Edge Cases

1. Recomputing maximum frequency every time

Wrong:

Scan all 26 values

inside loop

2. Incorrect replacement formula

Wrong:

$\text{maxFreq} - \text{windowLength}$

Correct:

$\text{windowLength} - \text{maxFreq}$

3. Forgetting to shrink invalid window

4. Missing:

$k=0$

case

12. Detailed Dry Run (Step-by-Step Table)

Input:

$s = \text{"AABABBA"}$

$k=1$

Window	Length	Max Frequency	Replacements Needed	Valid
A	1	1	0	Yes
AA	2	2	0	Yes
AAB	3	2	1	Yes
AABA	4	3	1	Yes
AABAB	5	3	2	No

Shrink window.

Maximum valid:

4

13. Common Use Cases (Real-Life / Interview)

Signal correction

Text normalization



DNA mutation analysis

Pattern matching systems

14. Common Traps (Important!)

1. Wrong sliding window condition
2. Updating max frequency incorrectly
3. Forgetting window shrink
4. Using nested loops

15. Builds To (Related LeetCode Problems)

Progression:

- LC 424 → Longest Repeating Character Replacement
- LC 3 → Longest Substring Without Repeating Characters
- LC 76 → Minimum Window Substring
- LC 567 → Permutation in String
- LC 438 → Find All Anagrams in a String

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
Sliding Window + HashMap	$O(n)$	$O(1)$	Good
Sliding Window + Array	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

The most frequent character in a window does not need replacement. Only the remaining characters need changes:

$\text{windowLength} - \text{maxFrequency}$

If replacements needed stay within k , the window remains valid.

18. Variations / Follow-Up Questions

1. Return substring instead of length
2. Support lowercase + uppercase letters
3. Allow different replacement costs
4. Find minimum replacements required
5. Solve for streaming data



6.